

## Relatório Atividade 2 - Algoritmos de Ordenação.

### Código Principal:

```
1  using System;
2  using System.IO;
3  using System.Collections.Generic;
4  using System.Timers;
5
6  namespace Atividade_02
7  {
8      class Program
9      {
10         static async Task Main(string[] args)
11         {
12             string[] weapons = File.ReadAllLines("./weapons.txt"); // Transforma o arquivo TXT em um string com as definições de cada arma (nome, raridade e dano).
13             List<Armas> armas = new List<Armas>(); // Salva cada arma do TXT como um objeto nesta lista.
14
15             Print.msgCritério();
16             int selCritério = int.Parse(Console.ReadLine());
17             Print.msgAlgoritmo();
18             int selAlgoritmo = int.Parse(Console.ReadLine());
19
20             Print.listaInteira(armas, "Desordenado");
21
22             // Verifica as linhas do TXT de 3 em 3, por conta de a cada 3 linhas ter as definições das armas.
23             for (int i = 0; i + 2 < weapons.Length; i += 3)
24             {
25                 // Ao invés de precisar fazer contas para saber onde cada definição está, apenas fazemos o iterador 'i' pular os espaços com o incremento necessário.
26                 armas.Add(new Armas(weapons[i], weapons[i + 1], int.Parse(weapons[i + 2])));
27
28             }
29
30             Func<Armas, Armas, bool> comparador = selCritério switch
31             {
32                 1 => (a, b) => CompareArmas.Nome(a, b),
33                 2 => (a, b) => CompareArmas.Raridade(a, b),
34                 3 => (a, b) => CompareArmas.Dano(a, b),
35                 _ => (a, b) => CompareArmas.Dano(a, b)
36             };
37
38             switch (selAlgoritmo)
39             {
40                 case 1:
41                     Sort.Merge(armas, comparador);
42                     break;
43                 case 2:
44                     Sort.Quick(armas, 0, armas.Count - 1, comparador);
45                     break;
46                 case 3:
47                     Sort.Selection(armas, comparador);
48                     break;
49                 _:
50                     Sort.Merge(armas, comparador);
51                     break;
52             }
53
54             Print.listaInteira(armas, "Ordenado");
55         }
56     }
```

Esse é o código principal, o qual roda o programa. É aqui que escolhemos de que forma queremos ordenar as armas disponíveis.

Esse código pega o arquivo txt, transforma os elementos dele em strings com as definições de cada arma, salvando cada arma como um objeto na lista. Ele verifica as linhas no txt de 3 em 3, devido às definições de cada arma. Após isso, ao invés de fazer contas para saber onde cada definição está, fazemos o iterador "i" pular o espaço com o incremento necessário, como "i+2".

Também atribuímos cada forma de se organizar à um número:

- 1 compara por nome;
- 2 compara por raridade;
- 3 compara por dano;
- Caso alguém coloque qualquer outro valor, ao invés do programa crashar, será automaticamente organizado a partir do dano.

Após isso, atribuímos:

- 1 ao Merge;
- 2 ao Quick;
- 3 ao Selection;
- (any) ao Merge.

### Código de Prints:

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4  using static System.Runtime.InteropServices.JavaScript.JSType;
5
6  namespace Atividade_02
7  {
8      internal class Print
9      {
10         private const string DIVISOR = " ----- ";
11         public static void divisor()
12         {
13             Console.WriteLine(DIVISOR);
14         }
15         public static void inicio(string lista)
16         {
17             divisor();
18             Console.WriteLine($" -> Lista {lista}:");
19         }
20         public static void elemento(string elemento)
21         {
22             Console.WriteLine($" -> {elemento}");
23         }
24         public static void fim()
25         {
26             divisor();
27         }
28     }
29 }
```

```

29  ✓      public static void listaInteira<T>(List<T> lista, string titulo)
30      {
31          Console.WriteLine($"----- Lista {titulo} -----");
32          foreach (T arma in lista) {
33              Console.WriteLine((arma as Armas)?.getValores());
34          }
35          divisor();
36      }
37
38  ✓      public static void msgCritério()
39      {
40          Console.WriteLine("----- Favor escolha uma das opções ----- ");
41          Console.WriteLine(" 1 - Ordenar por ordem alfabética ");
42          Console.WriteLine(" 2 - Ordenar por raridade ");
43          Console.WriteLine(" 3 - Ordenar por dano ");
44          Console.WriteLine("");
45      }
46
47  ✓      public static void msgAlgoritmo()
48      {
49          Console.WriteLine("----- Favor escolha uma das opções ----- ");
50          Console.WriteLine(" 1 - Utilizar Merge Sort ");
51          Console.WriteLine(" 2 - Utilizar Quick Sort ");
52          Console.WriteLine(" 3 - Utilizar Selection ");
53          Console.WriteLine("");
54      }
55
56      public static void finalizado_sort()
57      {
58          Console.WriteLine("Sort concluído");

```

Essa parte é a pasta “copia e cola” dos prints.

Para não precisarmos escrever os prints todas as vezes, deixamos eles separados. Assim é só puxar o método, facilitando o código e o deixando mais organizado.

## Código dos Sorts:

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5  namespace Atividade_02
6  {
7      internal class Sort
8      {
9          private static void Swap<T>(List<T> lista, int index1, int index2)
10         {
11             T aux = lista[index1];
12             lista[index1] = lista[index2];
13             lista[index2] = aux;
14         }
15         public static void Merge<T>(List<T> lista, Func<T, T, bool> compare)
16         {
17             // Caso a lista for menor que 2 elementos, não irá executar a função inteiramente.
18             if (lista.Count <= 1)
19             {
20                 return;
21             }
22             // Variáveis para irmos dividindo a lista inicial.
23             int metadeLista = lista.Count / 2;
24
25             List<T> listaEsq = new List<T>();
26             List<T> listaDir = new List<T>();
27
28             Console.WriteLine("Separando a lista");
29             // Adiciona os elementos da lista ESQUERDA até o limite definido -> (metadeLista).
30             for (int i = 0; i < metadeLista; i++)
31             {
32                 listaEsq.Insert(i, lista[i]); // Coloca o elemento da lista na lista esquerda.
33             }
34
35             Print.fim();
36
37             // Adiciona os elementos da lista DIREITA até o limite definido -> (lista.Count - metadeLista).
38             for (int i = 0; i < lista.Count - metadeLista; i++)
39             {
40                 listaDir.Insert(i, lista[i + metadeLista]); // Coloca o elemento da lista na lista direita.
41             }
42             Print.fim();
43
44             // Divide as listas até termos listas com apenas 1 elemento.
45             Merge(listaEsq, compare);
46             Merge(listaDir, compare);
47
48             // Verifica todos os elementos, agora divididos, para colocarmos em ordem.
49             int indFinal = 0; // Índice da lista final para podermos avançar e adicionar os elementos ordenados.
50
51             int indEsq = 0; // Índice da lista esquerda para podermos avançar entre a lista esquerda sem perder o último elemento da listaDir.
52             int indDir = 0; // Índice da lista direita para podermos avançar entre a lista direita sem perder o último elemento da listaEsq.
53
54             // Enquanto os índices das listas esquerda e direita forem menores que o tamanho das listas, iremos comparar os elementos.
55             while (indEsq < listaEsq.Count && indDir < listaDir.Count)
56             {
57                 // Comparamos elementos das 2 listas para colocar na lista completa.
58                 if (compare(listaEsq[indEsq], listaDir[indDir])) // Se o elemento da lista esquerda for menor, irá colocar ela no início da lista final e iremos pa
59                 { lista[indFinal] = listaEsq[indEsq++]; }
60                 else // Se não, irá colocar o elemento da lista DIREITA na lista final e passaremos para o próximo elemento da lista DIREITA.
61                 { lista[indFinal] = listaDir[indDir++]; }
62                 indFinal++;
63             }
64         }
65     }
66 }
```

```

64         // Por fim, copiamos quaisquer elementos restantes que "sobraram" nas duas partes
65         while (indEsq < listaEsq.Count)
66         {
67             lista[indFinal++] = listaEsq[indEsq++];
68         }
69         while (indDir < listaDir.Count)
70         {
71             lista[indFinal++] = listaDir[indDir++];
72         }
73     }
74
75     public static void Quick<T>(List<T> lista, int inicio, int fim, Func<T, T, bool> compare)
76     {
77         // Mesma lógica do Merge Sort, onde quando uma lista tiver apenas 1 elemento, iríamos parar de ordenar.
78         if (inicio >= fim)
79         {
80             return;
81         }
82
83         // O pivô é selecionado através do cálculo inicio + fim / 2, pois daí iríamos levar em consideração a diferença entre posição
84         // dos elementos de um array comparado com o tamanho do array.
85         int meio = (inicio + fim) / 2;
86         T pivo = lista[meio];
87
88         // Cria um valor para mantermos a organização de qual elemento de cada lado da lista estamos verificando.
89         int i = inicio;
90         int j = fim;
91
92         while (i <= j)

```

```

94         // Encontra elemento maior ou igual ao pivô pela esquerda
95         while (compare(lista[i], pivo))
96             i++;
97
98         // Encontra elemento menor que o pivô pela direita
99         while (compare(pivo, lista[j]))
100             j--;
101
102         // Se encontrou dois elementos fora de ordem, troca eles de lugar com a função Swap
103         if (i <= j)
104         {
105             Swap(lista, i, j);
106             i++;
107             j--;
108         }
109     }
110
111     // Depois de fazer o Sort, verifica se os elementos iniciais (i e j) estão na posição correta, e faz com que comecemos o Quick
112     // Sort em uma posição diferente.
113     if (inicio < j)
114         Quick(lista, inicio, j, compare);
115     if (i < fim)
116         Quick(lista, i, fim, compare);
117 }
118
119     public static void Selection<T>(List<T> lista, Func<T, T, bool> compare)
120     {
121         for (int i = 0; i < lista.Count - 1; i++)
122         {

```

```

122     {
123         // Primeiro selecionamos o menor ou maior valor usando a função de comparação
124         int selected = i;
125         for (int j = i + 1; j < lista.Count; j++)
126         {
127             if (compare(lista[j], lista[selected]))
128             {
129                 selected = j;
130             }
131         }
132
133         // Depois, apenas fazendo a troca dos elementos, até "empurrarmos" todos os elementos menores do valor selecionado
134         // para a esquerda.
135         Swap(lista, i, selected);
136     }
137 }
138
139 public static void Selection<T>(List <T> lista) where T : IComparable
140 {
141     Selection(lista, (a, b) => a.CompareTo(b) < 0);
142 }
143 }
144 }

```

Aqui estão os códigos de cada Sort diferente utilizado.

#### Função Swap:

Presente em todos os sorts, ele é responsável por pegar um index e substituí-lo na lista.

#### Função Merge:

Quando a lista for menor que 2 elementos, ele para de ordenar.

Ele seleciona um pivô, que seria o limite definido, e a partir disso re-arranja os elementos à esquerda ou direita, baseado em seu valor.

Após essa simples ordenação, ele divide a lista até termos listas com 1 elemento.

Agora que os elementos estão divididos, ele verifica todos os elementos para colocá-los em ordem. Mas enquanto os índices da lista esquerda e direita forem menores que o tamanho da lista, ele continua comparando.

Isto feito, o código compara os dois elementos em cada lista, para podermos juntá-los na lista principal novamente. Ou seja, se o elemento da lista esquerda for menor, ele vai pro início da lista principal, assim, passamos para o próximo item da lista da esquerda, se não, passamos para a lista da direita. Por fim, copiamos quaisquer elementos restantes que sobraram nas duas partes.

#### Função Quick:

Quando a lista tiver apenas 1 elemento, ele para de ordenar, igual ao Merge.

Um pivô é selecionado a partir do cálculo “início + fim / 2”, assim levamos em consideração a diferença entre posições de um elemento do array comparado com o tamanho total.

Depois, ele cria um valor para organizarmos qual elemento de cada lado da lista estamos verificando.

Ele encontra elementos maiores ou iguais que o pivô da esquerda, e menores que o da direita. Se dois elementos fora de ordem são encontrados, eles são trocados de lugar com a função Swap.

Depois de fazer o Sort, verifica se os elementos iniciais estão na posição certa, podendo começar o Quick.

Função Selection:

Primeiramente, o menor e o maior valor são selecionados usando a função de comparação.

Após isso, o código faz a troca de elementos, até que todos os elementos menores sejam “empurrados” para a esquerda.

### **Complexidade de cada algoritmo:**

Podemos ver qual dos códigos são mais pesados e mais complexos vendo a quantidade de operações e até mesmo pelo tamanho do código.

O mais complexo nesse caso seria o Merge Sort, que apesar de eficiente como os outros, ele realiza mais operações para poder chegar no resultado final. (Tempo:  $O(n \log n)$  Espaço:  $O(n)$ )

Após isso, temos o Quick Sort, que ainda é eficiente e rápido, mas é um pouco mais eficiente do que o Merge.  $O(n \log n) - n^2$

Por último, temos o Selection Sort, que apesar de ser um pouco mais lento, ele é o mais simples e leve de todos, já que tem poucas operações, e operações simples, porém por ser rodado várias vezes, demora um pouco mais. Tempo:  $O(n^2)$ , Espaço  $O(1)$

### **Em caso de empate:**

No Merge e no Quick, quando ocorre empate na parte alfabética, a ordem é decidida com base na lista base, ou seja: O elemento que aparece primeiro na lista base, vai parar em último, entre os itens empatados.